UNIVERSITATEA „POLITEHNICA” din BUCUREȘTI

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect

Programarea Interfețelor pentru Baze de Date

(2)

Student: Vincene Eduard

Grupa: 433A

Prof. coordonator:

Ș.l. Dr. Ing. Pupezescu Valentin

Cuprins

1. Cerințe 3
2. Tehnologii utilizate 3
   2.1 MySQL.....3
   2.2 Python 4
   2.3. Flask 4
   2.4. HTML 4
3. Descrierea aplicației 5
   3.1 Baza de date 5
   3.2 Diagrama logică a bazei de date (Diagrama ERD) 6
4. Implementarea Logicii Backend (DBManager).....7
   4.1 Inițializarea și conectarea la baza de date 7
   4.2 Închiderea conexiunii 8
   4.3 Manipularea Datelor 8
5. Implementarea Interfeței Web.....9
   5.1. Vizualizarea Tabelară 9
   5.2. Adăugarea Datelor 10
   5.3. Modificarea Datelor 12
   5.4. Ștergerea Datelor 13
   5.5. Gestionarea Relației M:N (Încadrări) 14
6. Bibliografie.....15

1. Cerințe

Să se creeze două aplicații care să conțină o bază de date creată în sistemul de gestiune a bazelor de date MySQL și două interfețe la aceasta (baza de date este comună).

La crearea interfețelor se vor folosi două tehnologii (ex: JSP, Hibernate, JPA, .NET, Python etc.). Baza de date va fi compusă din tabelele: Filme și Categorii, cu asociere de tip M:N.

Interfețele realizate trebuie să permită utilizatorului să execute următoarele operații pe toate tabelele: vizualizare, adăugare, modificare și ștergere de date. Vizualizarea tabelelor de legătură va presupune vizualizarea datelor referite din celalte tabele.

Pentru tema primită, am ales 2 tehnologii diferite JSP și Python. În acest document, se va face referire la tehnologia Python Flask.

2. Tehnologii utilizate

2.1 MySQL

MySQL este un sistem de gestionare a bazelor de date relaționale open source care este utilizat în principal pentru aplicațiile online. MySQL poate crea și gestiona baze de date foarte utile (cum ar fi informații despre filme, inventar și multe altele), la fel ca alte sisteme, cum ar fi popularul Microsoft Access. În timp ce Microsoft Access, MySQL și alte sisteme de gestionare a bazelor de date servesc scopuri similare (de a găzdui datele), utilizarea diferă foarte mult. [1]

2.2 Python

Python Python este un limbaj de programare interpretat, apreciat pentru sintaxa clară și eficiență. În cadrul acestui proiect, este utilizat pentru partea de Backend, gestionând logica aplicației și conexiunea cu baza de date MySQL. [2]

2.3. Flask

Flask este un micro-framework web pentru Python, flexibil și ușor de configurat. Acesta folosește motorul de templating **Jinja2** pentru generarea dinamică a paginilor HTML și include propriul server web, simplificând procesul de dezvoltare [3]

2.4. HTML

Unul din primele elemente fundamentale ale WWW (World Wide Web) este HTML (Hypertext Markup Language), care descrie formatul primar în care documentele sunt distribuite și văzute pe Web. Multe din trăsăturile lui, cum ar fi independență față de platforma, structurarea formatării și legaturile hypertext, fac din el un foarte bun format pentru documentele Internet și Web. [4]

3. Descrierea aplicatiei

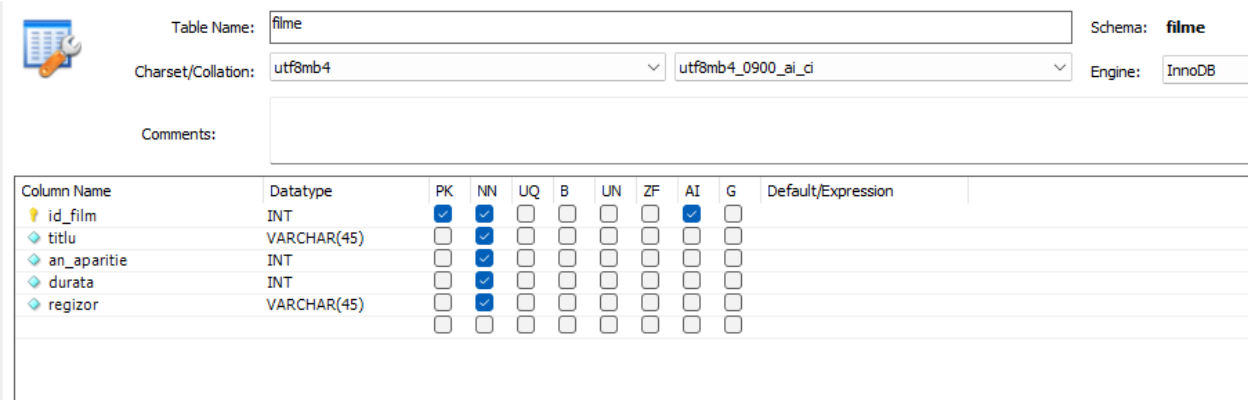
3.1 Baza de date

Tema individuală se bazează pe crearea unei baze de date ce are 2 tabele în asociere M:N. Tabelele sunt: filme si categorii.

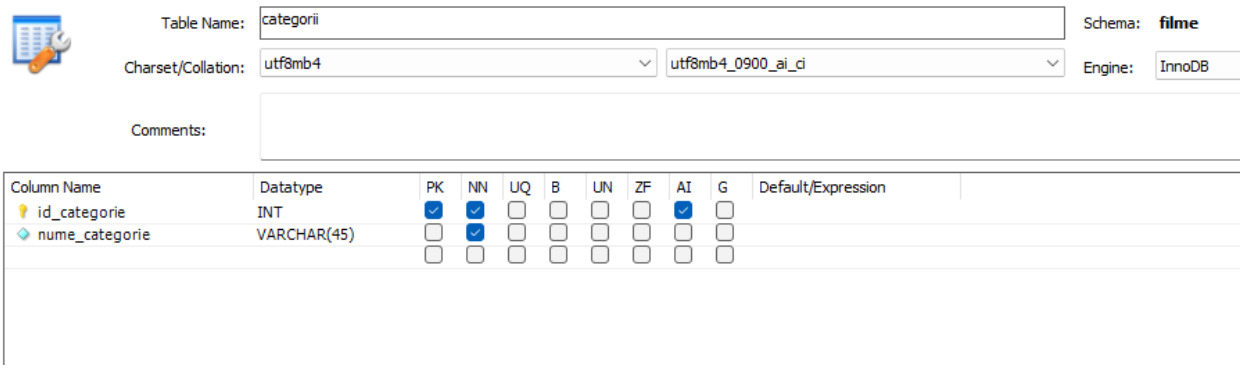
Asocierea M:N (Many-to-Many) are ca și caracteristică faptul că fiecărui element înregistrat într-o tabelă îi pot fi asociate mai multe elemente din cealaltă tabelă și invers. De exemplu, în acest caz, un film poate fi asociat mai multor categorii , așa cum și unei categorii îi pot fi asociate mai multe filme.

Pentru cele 2 tabele am ales cateva atribute caracteristice:

Pentru tabela filme: am ales ca și cheie primară id_film. Celelalte atribute sunt: titlu, an_aparitie, durata, regizor



Pentru tabela categorii: am ales ca si cheie primara id_categorie. Celalalt atribut este nume_categorie



Pentru a crea o relație M:N, trebuie să se creeze o a treia tabelă denumită tabelă de joctiune, care împarte relația mai-mulți-la-mai-mulți în două relații unu-la-mai-mulți. Am ales ca și tabela de legatură tabela incadrare_filme. În această nouă tabelă, atributele ce au fost selectate ca și chei primare pentru tabelele anterioare vor deveni chei străine (FK) pentru tabela de legatură incadrare_filme.

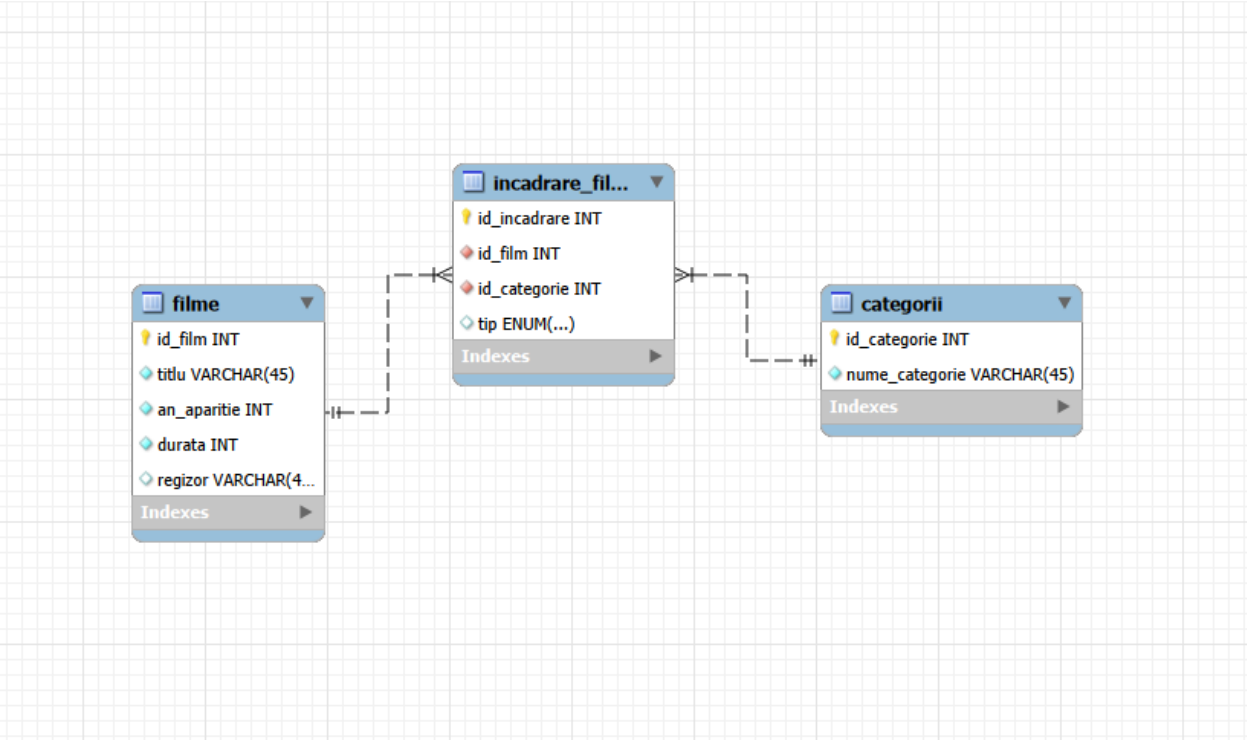
Pentru această tabelă, am ales atributul id_incadrare ca și cheie primară. Restul atributelor sunt cheile străine id_film și id_categorie și atributul ce determină tip-ul

Table Name: incadrare_filme
Schema: filme
Charset/Collation: utf8mb4 utf8mb4_0900_ai_ci
Engine: InnoDB
Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id_incadrare	INT	☑	☑	☐	☐	☐	☐	☑	☐	
id_film	INT	☐	☑	☐	☐	☐	☐	☐	☐	
id_categorie	INT	☐	☑	☐	☐	☐	☐	☐	☐	
tip	ENUM('Principala', 'Secundara')	☐	☐	☐	☐	☐	☐	☐	☐	'Principala'
		☐	☐	☐	☐	☐	☐	☐	☐	

3.2 Diagrama logică a bazei de date (Diagrama ERD)

Diagrama ERD este o hartă vizuală a bazei de date. Aceasta ne arată tabelele proiectului și liniile care le unesc, ajutându-ne să înțelegem rapid structura logică și modul în care datele comunică între ele



Asocierile dintre tabele:

- Între filme și categorii este o asociere M:N
- Între filme și incadrare_filme este o asociere 1:N
- Între categorii și incadrare_filme este o asociere 1:N

4. Implementarea Logicii Backend (DBManager)

Gestionarea interacțiunii cu baza de date este centralizată într-o clasă dedicată, numită **DBManager**, definită în fișierul db_manager.py. Această clasă asigură modularitatea aplicației, separând codul de conectare SQL de logica rutelor web.

Clasa conține metode specifice pentru stabilirea conexiunii, executarea interogărilor și închiderea securizată a resurselor.

4.1. Inițializarea și conectarea la baza de date

Clasa DBManager este inițializată prin constructorul __init__, unde este definit un dicționar de configurare. Acesta conține credențialele necesare accesului la serverul MySQL: utilizator, parolă, host și numele bazei de date (filme).

Metoda connect() utilizează biblioteca mysql.connector pentru a stabili o conexiune activă cu serverul, pe baza configurației definite anterior. Această metodă este apelată intern înainte de orice operațiune asupra datelor.

```
class DBManager:  2 usages
    def __init__(self):
        self.config = {
            'user': 'root',
            'password': 'mere22',
            'host': 'localhost',
            'database': 'filme',
            'raise_on_warnings': True
        }
        self.conn = None

    def connect(self):  3 usages
        """Deschide conexiunea"""
        self.conn = mysql.connector.connect(**self.config)
        return self.conn
```

4.2. Închiderea conexiunii

Managementul resurselor este asigurat de metoda disconnect(). Aceasta verifică existența unei conexiuni active și o închide folosind metoda .close()

```
def disconnect(self):  3 usages
    """Inchide conexiunea"""
    if self.conn and self.conn.is_connected():
        self.conn.close()
```

4.3. Manipularea Datelor

Metoda select(query, params): Este utilizată pentru operațiunile de citire a datelor (READ). Metoda deschide conexiunea, creează un cursor configurat să returneze rezultatele sub formă de **dicționar** (dictionary=True), execută interogarea și returnează toate rândurile găsite (fetchall). Returnarea datelor ca dicționare permite accesarea ulterioară a valorilor prin numele coloanei (ex: film['titlu']).

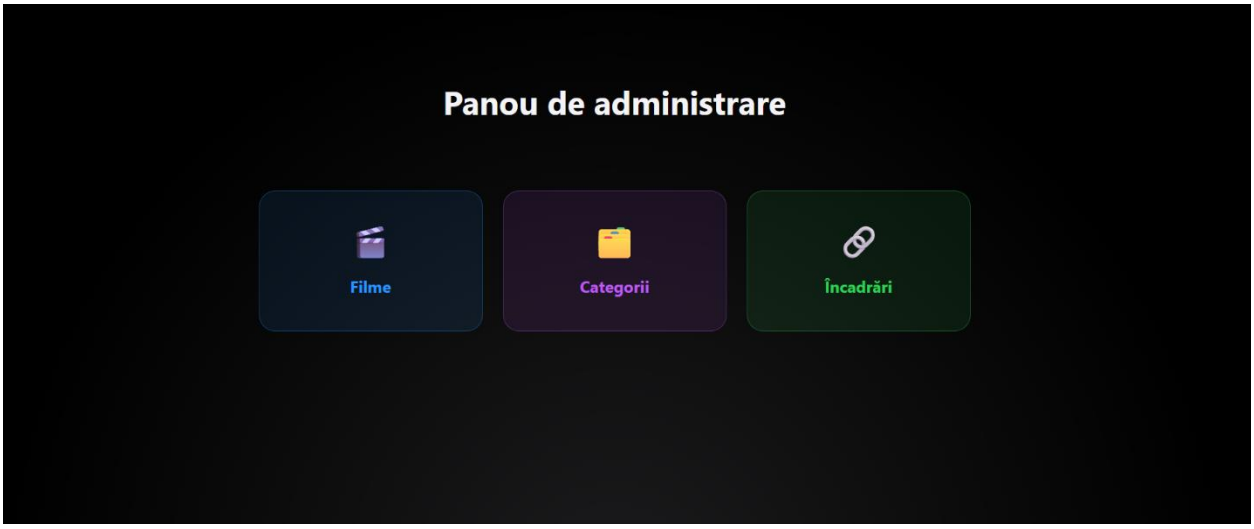
Metoda execute(query, params): Este utilizată pentru operațiunile de modificare a stării bazei de date (INSERT, UPDATE, DELETE). Aceasta execută interogarea și apelează obligatoriu metoda .commit() pentru a salva modificările permanent pe disc.

```
def select(self, query, params=None):  7 usages
    """Pentru SELECT-uri care returneaza mai multe randuri (tabele)"""
    self.connect()
    cursor = self.conn.cursor(dictionary=True) # Returneaza datele ca dictionar {'coloana': valoare}
    cursor.execute(query, params or ())
    result = cursor.fetchall()
    cursor.close()
    self.disconnect()
    return result

def select_one(self, query, params):  3 usages
    """Pentru SELECT-uri care returneaza un singur rand (modificare)"""
    self.connect()
    cursor = self.conn.cursor(dictionary=True)
    cursor.execute(query, params)
    result = cursor.fetchone()
    cursor.close()
    self.disconnect()
    return result

def execute(self, query, params):  12 usages (3 dynamic)
    """Pentru INSERT, UPDATE, DELETE"""
    self.connect()
    cursor = self.conn.cursor()
    cursor.execute(query, params)
    self.conn.commit() # Salvam modificarile
    cursor.close()
    self.disconnect()
```

5. Implementarea Interfeței Web

Interfața aplicației este construită pe baza framework-ului **Flask**, definită în fișierul app.py. Arhitectura respectă modelul MVC, unde app.py funcționează ca un Controller ce gestionează cererile HTTP și randează șabloanele HTML

Pagina principală este punctul de intrare în aplicație și afișează panoul de navigare către secțiunile principale: Filme, Categorii și Încadrări.

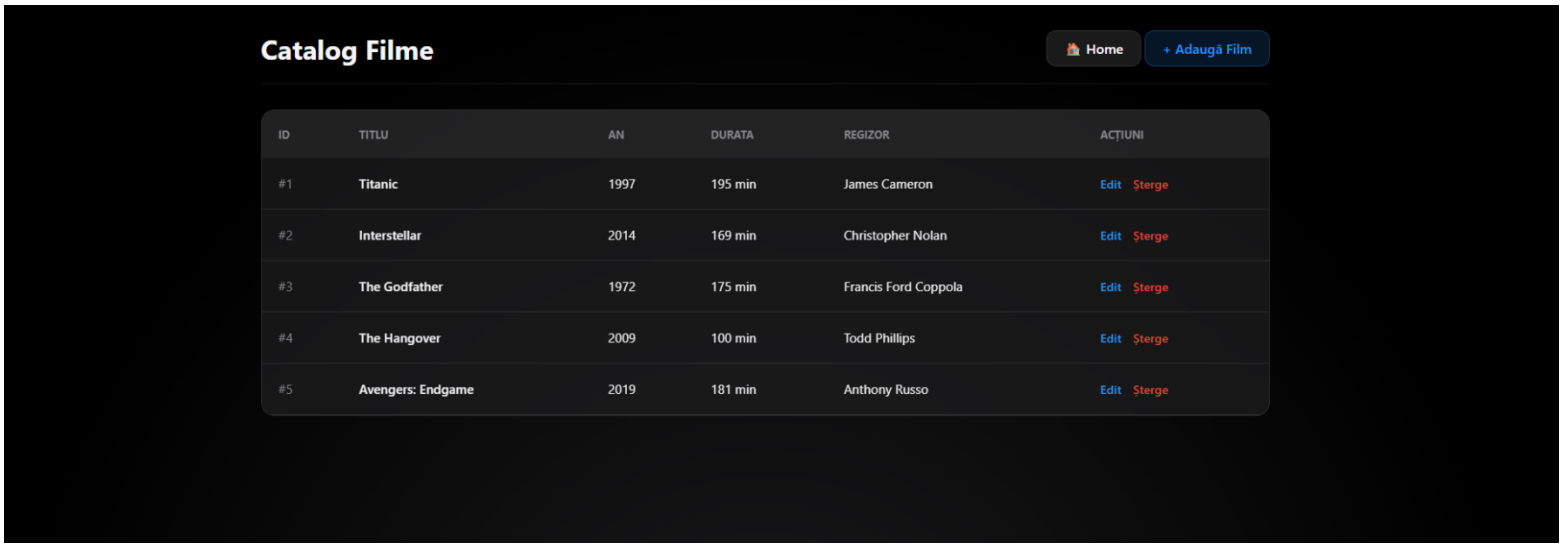


5.1. Vizualizarea Tabelară

Funcționalitatea de vizualizare este implementată în funcția filme(), asociată rutei /filme. Aplicația apelează metoda select din DBManager pentru a prelua toate înregistrările din tabelul filme.

Datele rezultate sunt trimise către șablonul HTML tabel_filme.html. Aici, motorul de templating **Jinja2** iterează prin lista de filme și generează dinamic rândurile tabelului pentru afișarea în browser.

```
# 1. Tabel Filme
@app.route('/filme')
def filme():
    data = db.select("SELECT * FROM filme")
    return render_template( template_name_or_list: 'tabel_filme.html', filme=data)
```

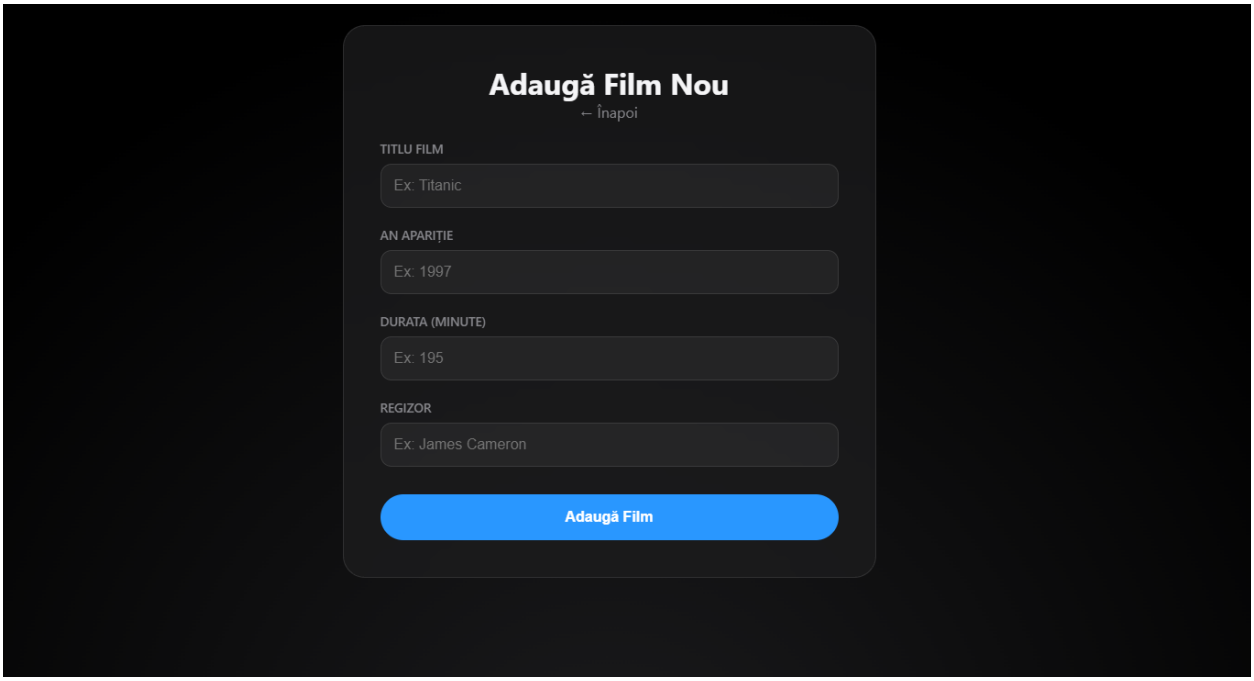


5.2. Adăugarea Datelor

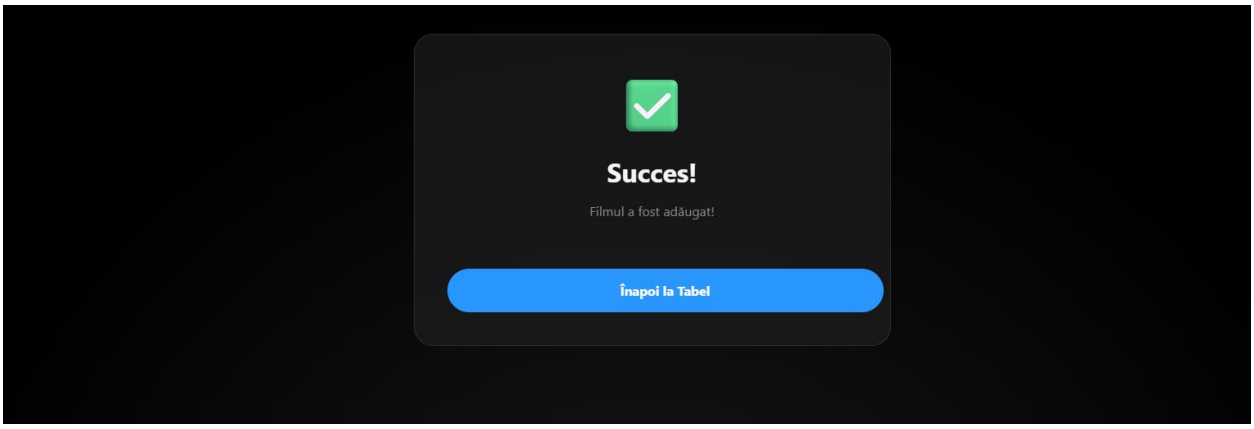
Procesul de adăugare a datelor este gestionat de ruta /filme/adauga, care acceptă două metode HTTP:

1. **GET:** Serverul randează formularul gol (adauga_filme.html) pentru a fi completat de utilizator.
2. **POST:** Serverul preia datele transmise prin formular (request.form), construiește o interogare parametrizată de tip INSERT și o execută prin intermediul instanței db.

```
# 2. Adauga Film
@app.route( rule: '/filme/adauga', methods=['GET', 'POST'])
def adauga_film():
    if request.method == 'POST':
        titlu = request.form['titlu']
        an = request.form['an_aparitie']
        durata = request.form['durata']
        regizor = request.form['regizor']

        sql = "INSERT INTO filme (titlu, an_aparitie, durata, regizor) VALUES (%s, %s, %s, %s)"
        db.execute(sql,  params: (titlu, an, durata, regizor))

        # Dupa adaugare, afisam pagina de succes
        return render_template( template_name_or_list: 'succes.html', mesaj="Filmul a fost adăugat!", link="/filme")

    return render_template('adauga_filme.html')
```



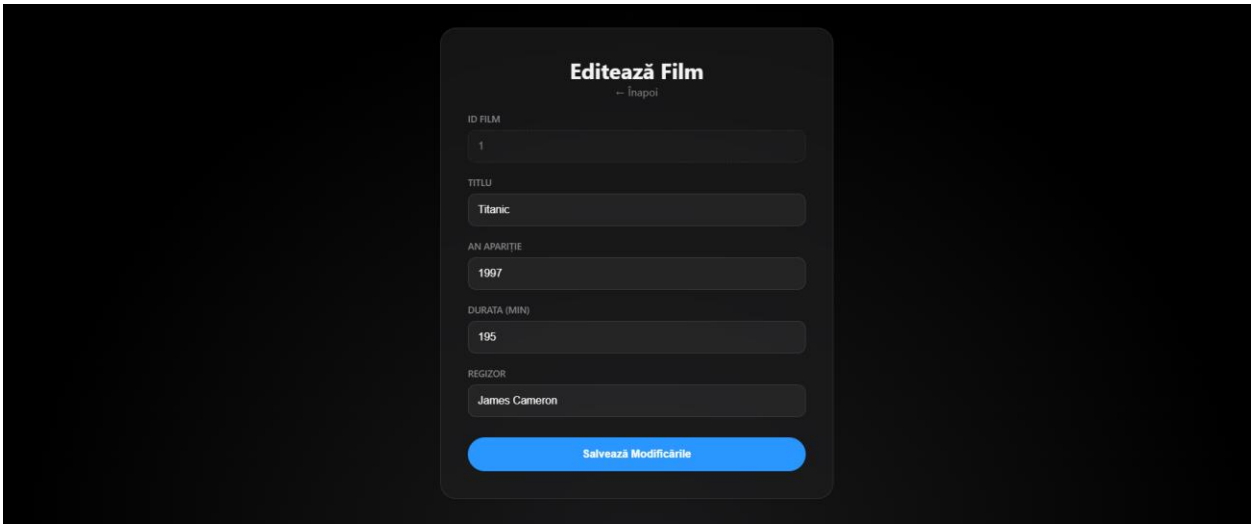
După inserare, utilizatorul este redirecționat către o pagină de succes.



5.3. Modificarea Datelor

Funcționalitatea de actualizare este implementată pe ruta /filme/modifica/<int:id_film>, care utilizează un parametru dinamic (ID-ul filmului).

- La accesarea prin **GET**, aplicația interoghează baza de date pentru a obține detaliile filmului selectat și pre-completează câmpurile formularului.
- La trimiterea prin **POST**, aplicația preia noile valori și execută o comandă UPDATE pentru înregistrarea respectivă.




```python
@app.route( rule: '/filme/modifica/<int:id_film>', methods=['GET', 'POST'])
def modifica_film(id_film):
    if request.method == 'POST':
        titlu = request.form['titlu']
        an = request.form['an_aparitie']
        durata = request.form['durata']
        regizor = request.form['regizor']

        sql = "UPDATE filme SET titlu=%s, an_aparitie=%s, durata=%s, regizor=%s WHERE id_film=%s"
        db.execute(sql,  params: (titlu, an, durata, regizor, id_film))

        return render_template( template_name_or_list: 'succes.html', mesaj="Filmul a fost modificat!", link="/filme")

    else:
        film = db.select_one( query: "SELECT * FROM filme WHERE id_film = %s",  params: (id_film,))
        return render_template( template_name_or_list: 'modifica_filme.html', film=film)
```

5.4. Ștergerea Datelor

Procesul este gestionat de ruta /filme/sterge (și echivalentele pentru celelalte tabele), care acceptă metoda **POST**.

1. Utilizatorul selectează rândurile dorite din interfață (checkbox-uri) și apasă butonul „Șterge Selecția”.
2. Formularul HTML trimite către server o listă de ID-uri selectate (request.form.getlist).
3. Serverul Flask iterează prin această listă și execută comanda SQL DELETE pentru fiecare ID identificat.

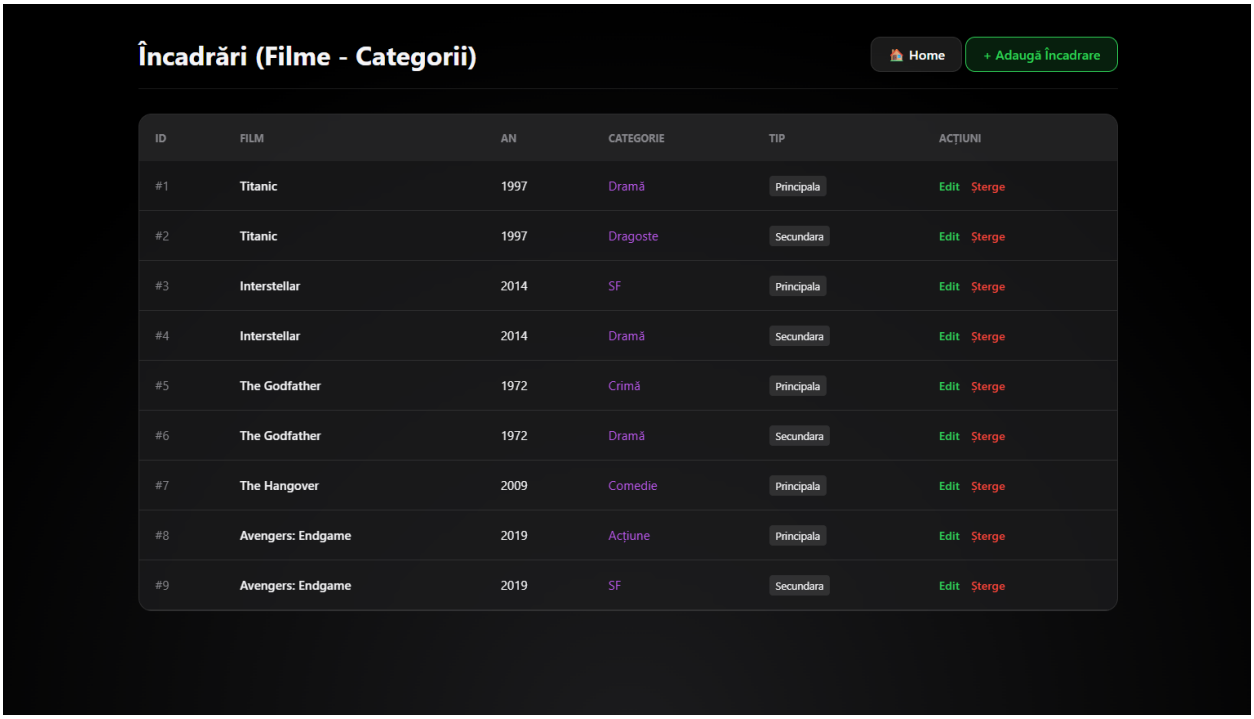
```
@app.route('/filme/sterge/<int:id_film>')
def sterge_film(id_film):
    db.execute( query: "DELETE FROM filme WHERE id_film=%s",  params: (id_film,))
    return redirect(url_for('filme'))
```

5.5. Gestionarea Relației M:N (Încadrări)

Pentru secțiunea de Încadrări, aplicația gestionează o interogare complexă de tip JOIN între trei tabele (incadrare_filme, filme, categorii). Aceasta permite afișarea numelor reale ale filmelor și categoriilor în loc de ID-urile numerice, facilitând citirea informației.

La adăugarea unei încadrări noi, controller-ul furnizează interfeței listele complete de filme și categorii, permițând utilizatorului să selecteze valorile din liste derulante (dropdown-uri), asigurând astfel integritatea datelor introduse.

```
@app.route('/incadrari')
def incadrari():
    sql = """
        SELECT i.id_incadrare, f.titlu, f.an_aparitie, c.nume_categorie, i.tip
        FROM incadrare_filme i
        JOIN filme f ON i.id_film = f.id_film
        JOIN categorii c ON i.id_categorie = c.id_categorie
    """
    data = db.select(sql)
    return render_template( template_name_or_list: 'tabel_incadrari.html', incadrari=data)
```



6. Bibliografie

[1] - https://www.nav.ro/blog/ce-este-mysql/

[2] - https://www.python.org/doc/essays/blurb/

[3] - https://flask.palletsprojects.com/

[4] - https://web.ceiti.md/lesson.php?id=1